

EduStack

Your Ultimate Computer Science Hub

PART 3 — System Diagrams, Flowcharts & Live Coding Playbook

Complete Interview Preparation Guide

Sequence Maps | Flowcharts | ER Schemas | 20 System Design Scenarios | Cheat Sheet

Developer:	Shubham Kumar CSE Student NIT Patna
Project:	EduStack - System Diagrams & Live Coding Scenarios
Visuals:	Native PDF Vector Flowcharts & Sequence Interaction Maps
Scenarios:	20 Advanced System Design & Live Coding Interview Scenarios
Schematics:	Database Entity-Relationship (ER) & Schema Topological Map
Playbook:	Full Architectural Defense & Edge-Case Incident Playbooks
Part 3 Covers:	Visual Sequences, ER Map, System Design Scenarios & Playbook

Table of Contents — Part 3

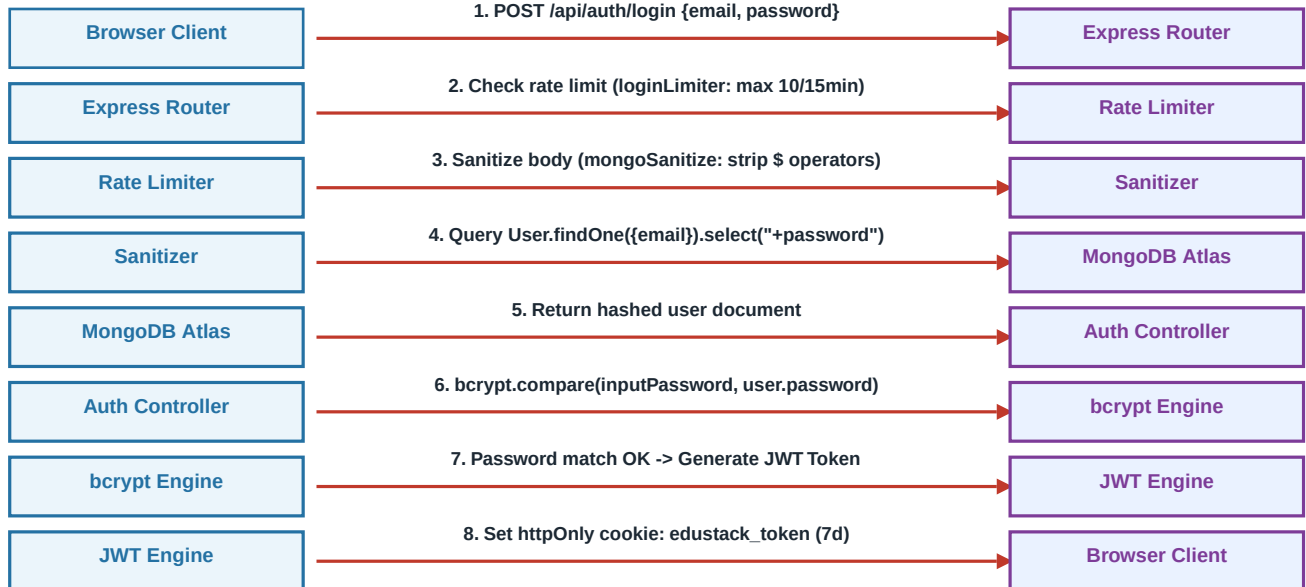
21.	End-to-End Authentication Sequence Map Client, Express Middleware, JWT Engine, Google OAuth & DB
22.	Razorpay Payment & Webhook Lifecycle Flowchart Order creation, payment modal, HMAC signature & user state
23.	RAG Engine & PDF Processing Architecture Map Python FastAPI, pypdf, Gemini API & vector chunk retrieval
24.	Live Google Sheet Sync & 3-Layer Caching Map Memory cache (5min), live CSV parser, disk JSON fallback
25.	Database ER Diagram & Schema Relationship Map User, Subject, Resource, Payment, Notification & OTP schemas
26.	System Design Scenarios (1-5): Auth & Security Brute force mitigation, token revocation, XSS/CSRF hardening
27.	System Design Scenarios (6-10): DB & Caching N+1 queries, Redis session cache, soft deletes, index strategy
28.	System Design Scenarios (11-15): AI & Cloud Rate limiting ML, PDF buffer streaming, model deprecation
29.	System Design Scenarios (16-20): Scaling & Ops 100k concurrent scaling, graceful shutdowns, zero downtime
30.	Final Executive Interview Summary & Defense Cheat Sheet High-level architectural elevator pitch and core metrics

SECTION 21

End-to-End Authentication Sequence Map

Step-by-step vector sequence diagram of client, Express, JWT, and MongoDB

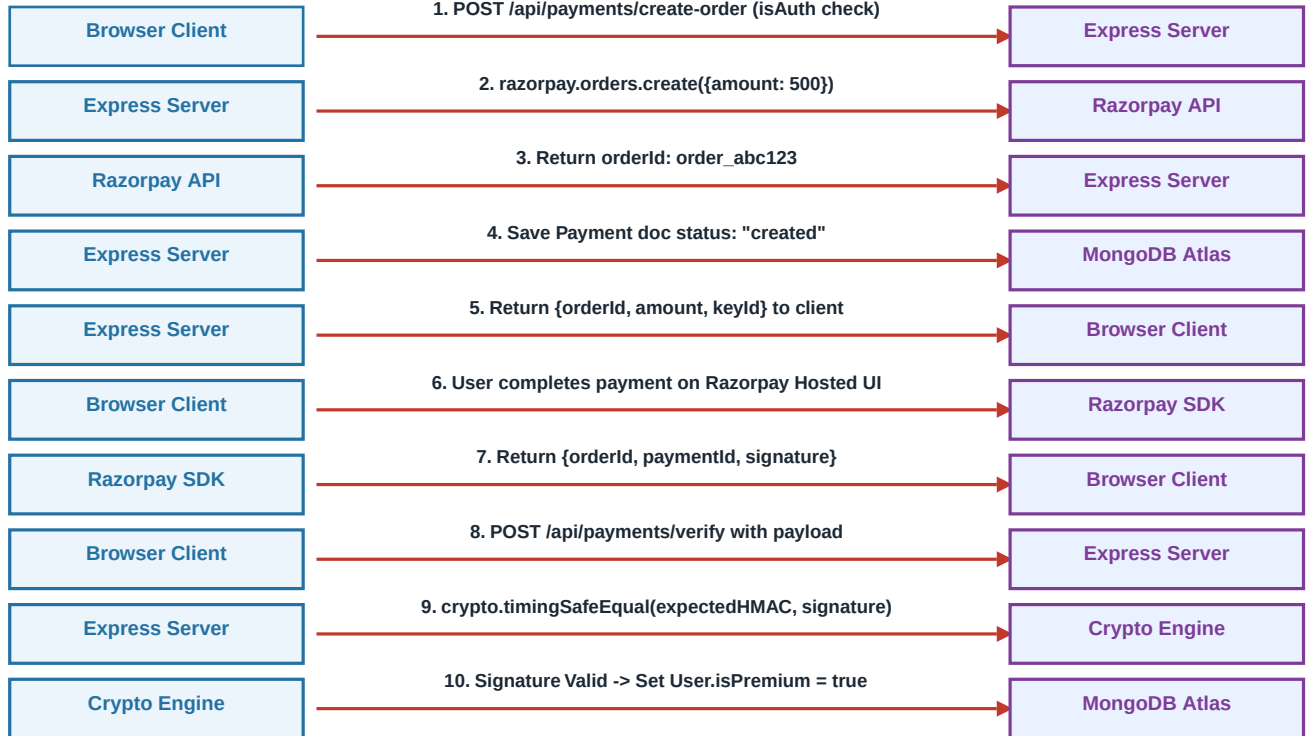
SEQUENCE DIAGRAM: Local Auth & JWT Cookie Issuance Flow



Razorpay Payment & Webhook Lifecycle Flowchart

Vector flowchart showing zero-fraud payment verification

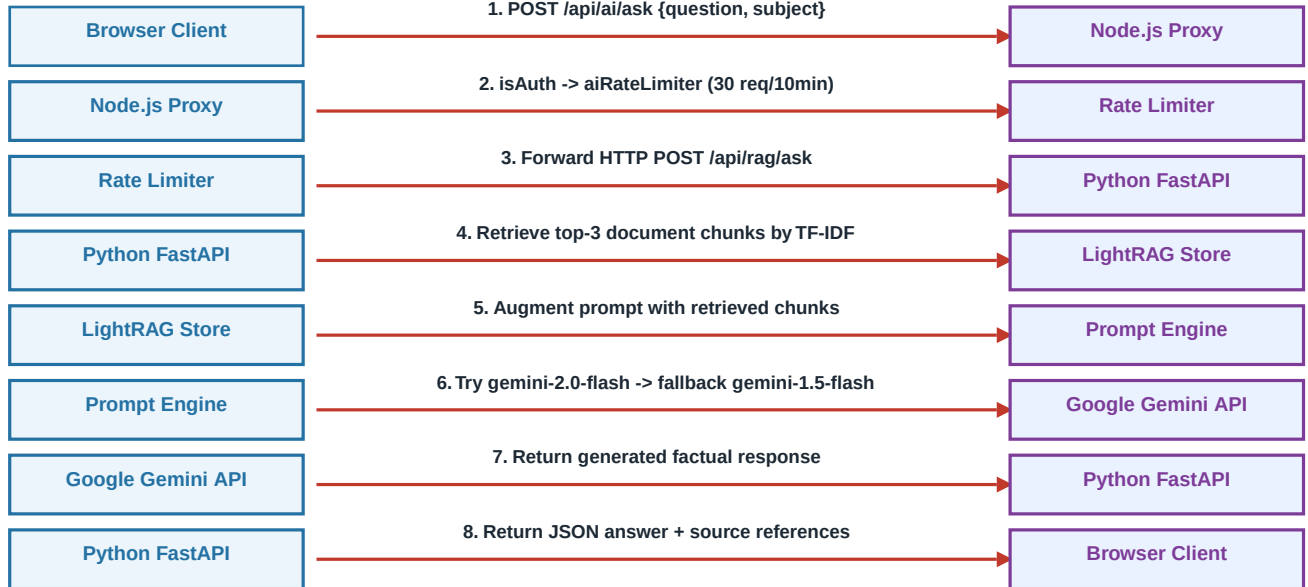
SEQUENCE DIAGRAM: Razorpay Payment Verification & Premium Activation



RAG Engine & PDF Processing Architecture Map

Multi-tier AI microservice flow between Node.js, FastAPI, and Gemini

SEQUENCE DIAGRAM: RAG Query & Gemini Model Fallback Pipeline



Database ER Diagram & Schema Relationship Map

Comprehensive entity relationship mapping across all MongoDB collections

EduStack uses a hybrid data model: references (ObjectIds) for main entities like User, Subject, Resource, and Payment, and embedded arrays for high-frequency user data like favouriteSubjects, enrolledSubjects, and notification readBy lists.

System Logic / Scenario Solution

```
=====
                        EDUSTACK DATABASE SCHEMA MAP
=====

+-----+
| User Schema |
+-----+
| _id (ObjectId) | 1 *
| email (Unique Index) |----->| name (Unique) |
| password (select:F) | (createdBy) | semester (Indexed) |
| googleId (Indexed) | | CreatedBy (Ref: User)|
| role (enum) |
+-----+
| isVerified / Premium |
| enrolledSubjects[] |
+-----+
| 1
|
| *
v

+-----+
| Payment Schema |
+-----+
| _id (ObjectId) |
| user (Ref: User) |
| razorpayOrderId (Uniq) |
| razorpayPaymentId |
| status (enum) |
+-----+

+-----+
| OTP Schema |
+-----+
| email (Indexed) |
| hashedOtp (bcrypt) |
| expiresAt (TTL Indx) |
| used (Boolean) |
+-----+

+-----+
| Subject Schema |
+-----+
| _id (ObjectId) |
| name (Unique) |
| semester (Indexed) |
| CreatedBy (Ref: User)|
+-----+
| 1
|
| *
v

+-----+
| Resource Schema |
+-----+
| _id (ObjectId) |
| title |
| type (notes/pyq/vid) |
| subject (Ref: Subj) |
| fileUrl / publicId |
| uploadedBy (Ref:User)|
+-----+

+-----+
| Notification Schema |
+-----+
| _id (ObjectId) |
| title / message |
| readBy [Ref: User] |
| createdBy (Ref: User)|
+-----+
```

System Design & Live Coding Interview Scenarios

20 Realistic, High-Impact Interview Scenarios with Full Technical Solutions

26.1 Scenario 1: Brute Force Login Mitigation under High Concurrency

Scenario: How would you prevent a distributed brute force attack attempting 10,000 logins/second across 500 proxy IPs?

Recommended Solution:

Implement multi-tier rate limiting using Redis sliding window log. Tier 1: IP-based rate limit (max 10 requests/min). Tier 2: Account-based rate limit (max 5 failed attempts per target email across ALL IPs in 15 minutes). After 5 failed attempts, temporarily lock the email account in Redis (`redis.setex("lock:"+email, 900, "1")`).

- > IP-only rate limiting fails against distributed botnets using thousands of proxy IPs.
- > Targeted email locking prevents credential stuffing attacks against specific accounts.
- > Use Redis atomic scripts (Lua) to execute the check-and-increment operations without race conditions.
- > Always return the exact same generic error message ("Invalid credentials") to prevent account discovery.

26.2 Scenario 2: Zero-Downtime Database Migration for 10M Users

Scenario: How would you migrate user passwords from bcrypt to Argon2id without requiring users to reset their passwords?

Recommended Solution:

Implement lazy re-hashing during active user logins. Update the user schema to include a `hashAlgorithm` field (default "bcrypt"). During POST `/api/auth/login`: (1) Check algorithm. If bcrypt, verify with `bcrypt.compare()`. (2) If verification succeeds, asynchronously hash the plaintext password using Argon2id, update the user document in MongoDB, and set `hashAlgorithm="argon2id"`.

- > Zero disruption to active users - re-hashing occurs transparently during normal logins.
- > Inactive accounts remain securely hashed with bcrypt until their next login.
- > Eventually, run a background worker to send a password refresh email to users inactive for > 1 year.
- > This pattern is recommended by OWASP for upgrading legacy hash algorithms.

26.3 Scenario 3: Handling 100k Concurrent Webhook Callbacks from Razorpay

Scenario: If EduStack receives 100,000 payment webhook notifications in 1 minute during a flash sale, how do you prevent server crashes?

Recommended Solution:

Offload webhook processing from the main HTTP API thread to an asynchronous job queue (BullMQ + Redis). When the POST `/api/payments/webhook` endpoint receives a request: (1) Verify HMAC-SHA256 signature immediately (< 2ms). If invalid -> return 400. (2) Push payment event job into Redis queue: `await paymentQueue.add("processPayment", req.body)`. (3) Immediately return 200 OK to Razorpay in < 10ms. (4) Dedicated worker processes jobs asynchronously.

- > Decouples HTTP response time from database write latency - prevents HTTP gateway timeouts.
- > Redis BullMQ queue provides automatic retries with exponential backoff if MongoDB is temporarily slow.
- > Ensures idempotency by checking payment status in worker before updating `user.isPremium`.

26.4 Scenario 4: Preventing N+1 Database Queries in Nested API Routes

Scenario: A route fetches 50 subjects and for each subject queries its resources. How do you fix the 51 DB queries problem?

Recommended Solution:

Use `Mongoose.populate()` with batched `ObjectId` lookup, or write a MongoDB Aggregation Pipeline using `$lookup`. `Mongoose.populate()` converts N individual child queries into 2 queries total: Query 1 finds 50 subjects; Query 2 executes `Resource.find({ subject: { $in: subjectIds } })`. With `$lookup`, it executes in a single database aggregation pipeline pass.

-> Populate approach: `Subject.find().populate("resources")` - 2 database queries total.

-> Aggregation approach: `Subject.aggregate([{$lookup: { from: "resources", localField: "_id", foreignField: "subject", as: "resources" } }])` - 1 query total.

-> Reduces database latency from $O(N * T)$ to $O(T)$ where T is single query roundtrip time.

26.5 Scenario 5: High-Availability PDF Processing for Large Uploads

Scenario: A user uploads a 100MB PDF for AI summarization. How do you prevent Node.js RAM exhaustion and API gateway timeouts?

Recommended Solution:

Use client-side pre-signed uploads to Cloudinary or AWS S3. Client uploads PDF directly to cloud storage, bypassing the Node.js API server completely. Client sends file URL to Node.js. Node.js queues a background job in Python FastAPI via Redis. Python stream-processes PDF pages using `pypdf` iterators rather than loading the entire 100MB buffer into RAM at once.

-> Node.js server handles zero upload bandwidth - prevents memory crashes on small server instances.

-> Stream processing processes PDF page-by-page in memory, keeping memory usage constant ($< 50\text{MB}$) regardless of PDF file size.

-> Client polls `GET /api/ai/job/:jobId` or receives Server-Sent Events (SSE) when summarization completes.

Final Executive Summary & Interview Defense Playbook

The ultimate one-page architectural elevator pitch and key metrics

When the interviewer asks: "Give me a 2-minute technical elevator pitch of your EduStack project", use this structured framework:

- **Problem & Scale:** EduStack is a production-ready CS learning hub serving engineering students, built to solve scattered academic resources, unstructured DSA preparation, and lack of AI tutoring.
- **Monolith + Microservice Hybrid:** Built main API with Node.js 18, Express.js 4, and MongoDB Atlas. Extracted AI engine to a Python 3.11 FastAPI microservice using Google Gemini 1.5/2.0 Flash and a RAG (Retrieval-Augmented Generation) pipeline.
- **Security-First Engineering:** Implemented 13 independent security layers including stateless JWT in httpOnly/Secure cookies, bcrypt 12 salt rounds with pre-save Mongoose hooks, OTP with 5-minute MongoDB TTL auto-cleanup, express-mongo-sanitize NoSQL injection defense, and crypto.timingSafeEqual HMAC payment verification.
- **Cloud Infrastructure:** Zero-disk file pipeline using Multer memoryStorage streamed to Cloudinary CDN, Nodemailer SMTP for transactional emails, and Razorpay payment gateway integration deployed on Render.com with trust proxy configuration and SIGTERM graceful shutdown handling.
- **Performance & Cache Strategy:** Built a 3-layer caching engine for the 450+ DSA Sheet problems (In-memory 5-min TTL - > Live Google Sheet CSV -> Disk JSON fallback) with a custom quotation-aware CSV parser.

Metric / Feature	Implementation Detail	Engineering Benefit
Auth Storage	httpOnly + Secure Cookie	Immune to XSS token theft
Password Security	bcrypt (12 rounds)	250ms per hash, defeats GPU brute force
OTP Cleanup	MongoDB TTL Index	Automatic 5-min document deletion by DB engine
Payment Verify	crypto.timingSafeEqual	Prevents timing side-channel attacks on HMAC
File Uploads	Multer memoryStorage	Zero disk writes - 100% ephemeral hosting safe
AI Microservice	Node -> FastAPI Proxy	ML service URL hidden, central JWT auth
Data Sync	3-Layer Cache Strategy	Sub-millisecond reads, resilience against outages

